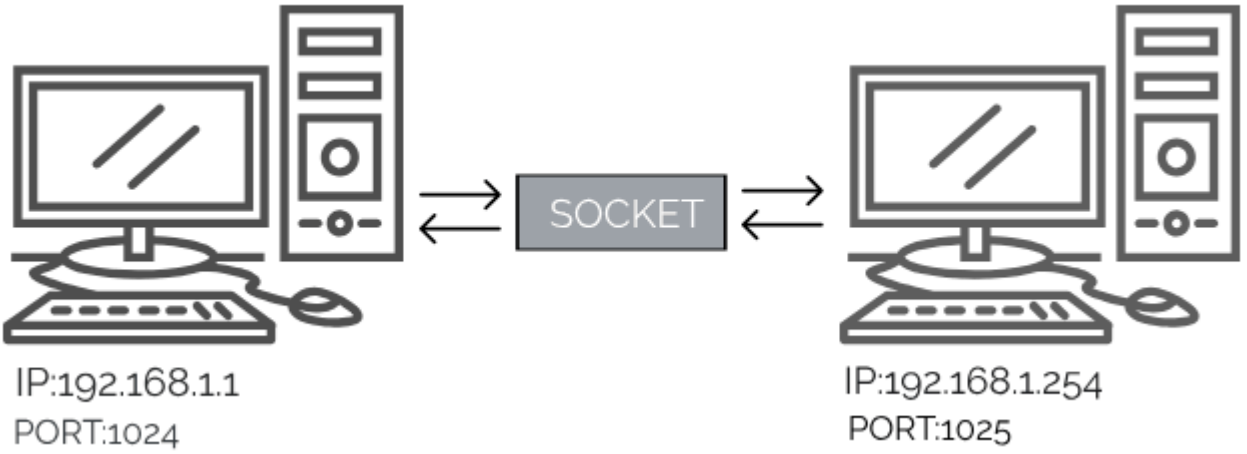
Socket Programlama

Bu bölümde, socket programlamanın temel kavramlarını ve iki önemli I/O (Giriş/Çıkış) modelini inceleyeceğiz. Ardından, bir önceki derste yaptığımız TCP chat uygulamasını UDP protokolü ile tekrar deneyerek iki protokol arasındaki farkları pratik olarak gözlemleyeceğiz.



1. Client-Server Mimarisi (Tekrar)

Client-Server mimarisi, ağ üzerindeki kaynakların ve hizmetlerin nasıl paylaşıldığını tanımlayan temel bir modeldir. Bu modeli daha detaylı inceleyelim:

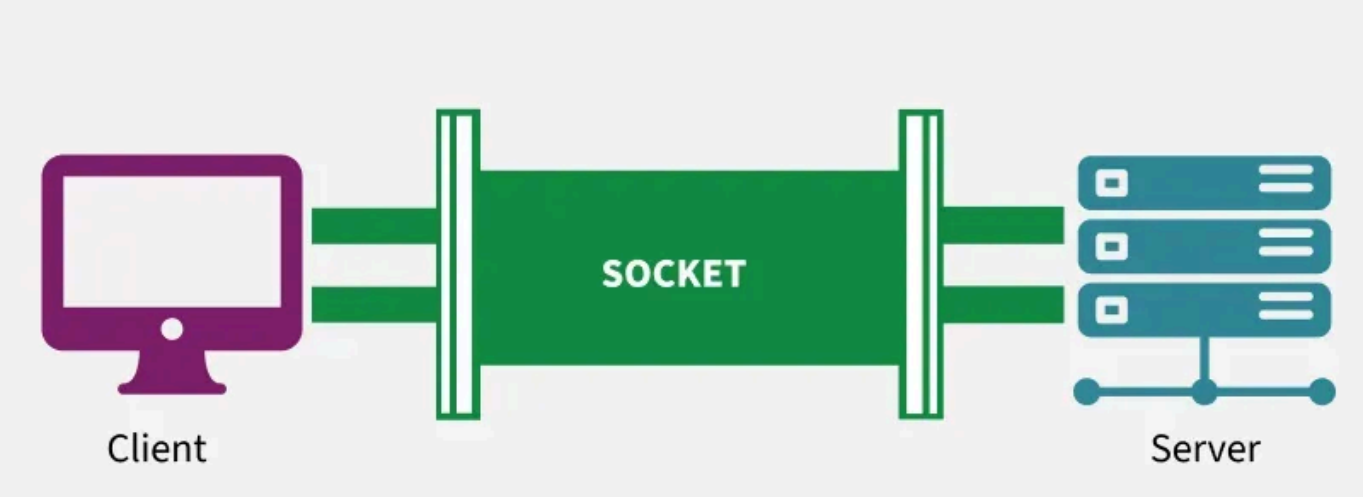
1.1. Sunucu (Server)

- **Tanım:** Hizmet sağlayan, kaynakları yöneten ve istemcilerin isteklerine yanıt veren sistemdir.
- **Özellikler:**
  - Sürekli çalışır (always-on) ve gelen bağlantıları dinler.
  - Genellikle sabit bir IP adresine ve port numarasına sahiptir.
  - Aynı anda birden fazla istemciye hizmet verebilir (multi-threading veya asenkron yapı ile).
  - Yetkili otoritedir (authoritative); tüm kritik kararları alır.

1.2. İstemci (Client)

- **Tanım:** Sunucunun sunduğu hizmetleri kullanan, istek gönderen sistemdir.
- **Özellikler:**
  - İhtiyaç duyduğunda sunucuya bağlanır (bağlantıyı başlatan taraftır).
  - Dinamik port numaraları kullanır (sistem tarafından atanır).
  - Birden fazla sunucuya aynı anda bağlanabilir.
  - Sunucuya bağlı olarak çalışır, sunucu olmadan tam işlevselliğini gösteremez.

1.3. İletişim Akışı (TCP)




```
[İstemci]                                    [Sunucu]
   |                                            |
   |------ SYN (Bağlantı isteği) ---------------->|
   |<----- SYN-ACK (Kabul) ----------------------|
   |------ ACK (Onay) ---------------------------->|  (3-Way Handshake)
   |                                            |
   |------ Veri Gönderimi (Mesaj) --------------->|
   |<----- Veri Yanıtı (Onay/State Update) -------|
   |                                            |
   |------ FIN/ACK (Bağlantıyı Kapat) ----------->|
   |<----- FIN/ACK (Kapanış Onayı) ---------------|
```

---

2. Blocking vs Non-Blocking I/O

I/O (Input/Output) işlemleri, programlamada kritik bir performans konusudur. Bir işlemin "blocking" veya "non-blocking" olması, o işlemin program akışını nasıl etkilediğini belirler.

2.1. Blocking I/O (Engelleyici Giriş/Çıkış)

- **Tanım:** Bir I/O işlemi çağrıldığında, işlem tamamlanana kadar (veri gelene veya bağlantı kurulana kadar) programın o noktada **beklemesi** (block) durumudur.
- **Nasıl Çalışır?** Program, `recv()` veya `accept()` gibi bir fonksiyonu çağırdığında, işletim sistemi bu fonksiyonu bloke eder. Veri gelene kadar CPU başka iş yapamaz (thread/process uyku moduna geçer).
- **Avantajları:** Kod yazması basittir, mantıksal akış kolay takip edilir.
- **Dezavantajları:** Verimlilik düşüktür. Tek bir thread, bir istemcinin veri göndermesini beklerken diğer istemcilere hizmet veremez.
- **Çözüm:** Her istemci için ayrı bir thread kullanmak (önceki TCP Chat örneğinde yaptığımız gibi). Ancak bu, çok sayıda istemci için kaynak tüketimini artırır.

```
// Blocking I/O Örneği (Pseudo)
clientSocket = accept(serverSocket);  // Burada bir bağlantı gelene kadar
BEKLER
```

```
data = recv(clientSocket);            // Veri gelene kadar BEKLER
process(data);
```

2.2. Non-Blocking I/O (Engellemeyen Giriş/Çıkış)

- **Tanım:** Bir I/O işlemi çağrıldığında, işlem hemen geri döner. Eğer veri hazır değilse, "henüz veri yok" (EWOULDBLOCK) hatası ile döner ve program çalışmaya devam eder.
- **Nasıl Çalışır?** Program, I/O işlemini çağırır, sonucu kontrol eder. Veri gelmemişse başka işlere devam eder ve daha sonra tekrar kontrol eder. Bu yöntemde program "polling" (sürekli kontrol etme) yapar.
- **Avantajları:** Tek bir thread ile birçok istemci aynı anda yönetilebilir (Event-Driven / Asenkron). Kaynak kullanımı daha verimlidir.
- **Dezavantajları:** Kod yazması daha karmaşıktır. Durum yönetimi ve veri bütünlüğü daha zordur.
- **Gelişmiş Çözüm:** `select`, `poll`, `epoll` (Linux) veya `IOCP` (Windows) gibi mekanizmalarla birden fazla soket tek bir thread'den izlenir.

```
// Non-Blocking I/O Örneği (Pseudo)
setNonBlocking(clientSocket);
while (true) {
    data = recv(clientSocket);
    if (data == EWOULDBLOCK) {
        // Veri yok, başka iş yap
        doOtherWork();
        continue;
    } else if (data > 0) {
        process(data);
    }
}
```

---

1. Socket detayları

Socket, kernel'de tutulan bir **dosya tanıtıcısı (fd)** ve ona bağlı bir **çekirdek nesnesidir**. İçinde bağlantı durumu, buffer ve protokol bilgileri bulunur.

Kritik sonuç: Socket = fd olduğu için `read()`/`write()` gibi dosya API'leri de çalışır, `select()`/`poll()`/`epoll()` ile izlenebilir.

---

2. Socket Yaşam Döngüsü

```
socket() → bind() → listen() → accept() → read/write → close()
```

Bilinmesi gereken durumlar:

- **LISTEN** — Sunucu bağlantı bekliyor

- **ESTABLISHED** — Bağlantı kurulu, veri akışı var
- **TIME_WAIT** — Aktif kapatma yapan taraf ~60 sn bekler (gecikmiş paketler karışmasın diye)
- **CLOSE_WAIT** — Karşı taraf kapattı, sen henüz `close()` çağırmadın

Önemli: `accept()` **yeni bir fd döner**. Dinleyen socket değil, o bağlantıya özel yeni bir socket.

---

3. Sunucu Mimarî Desenleri

Desen	Ne zaman?
Iterative (tek thread, sıralı)	Sadece öğretici örnekler
Thread-per-connection	100–1000 bağlantı, blocking API rahat
Event-driven (epoll/select)	10.000+ bağlantı, tek thread
Async I/O (asyncio)	Modern Python, event-driven üstüne kurulu

Temel kural: Bağlantı sayısı arttıkça thread'den event-driven'a geç.

---

4. Zorunlu Socket Seçenekleri

Seçenek	Neden?
`SO_REUSEADDR`	TIME_WAIT'teki port'a tekrar bind için
`TCP_NODELAY`	Nagle'ı kapat (Nagle algoritması, TCP/IP ağlarında küçük boyutlu veri paketlerini birleştirerek ağ trafiğini azaltan ve verimliliği artıran bir yöntemdir.) → düşük gecikme (oyun, SSH)
`SO_RCVBUF` / `SO_SNDBUF`	Buffer boyutları

Nagle + Delayed ACK tuzağı: İkisi birleşince 200 ms gecikme olur. Çözüm: `TCP_NODELAY`.

---

5. Backlog ve accept Kuyruğu

`listen(fd, backlog)` — `backlog` sadece öneri, kernel `somaxconn`'a göre kırpar.

İki kuyruk vardır:

1. **SYN queue** — SYN geldi, ACK bekleniyor
2. **Accept queue** — Handshake bitti, `accept()` bekleniyor

Kuyruk dolarsa → istemci "connection timeout" yaşar.

---

6. En Sık Yapılan Hata: Partial Read/Write

`recv()` tüm mesajı getirmez! Sadece "şu ana kadar gelen kadar" döner.

```
# YANLIŞ
data = sock.recv(1024)   # yarım gelmiş olabilir!

# DOĞRU — uzunluk önekli framing
def recv_exact(sock, n):
    buf = b""
    while len(buf) < n:
        chunk = sock.recv(n - len(buf))
        if not chunk:
            raise ConnectionError("bağlantı kapandı")
        buf += chunk
    return buf

def recv_message(sock):
    length = struct.unpack("!I", recv_exact(sock, 4))[0]
    return recv_exact(sock, length)
```

Aynı şey `send()` için de geçerli: `send(1024)` = "1024 byte'a kadar kabul edildi". `sendall()` bunu döngüye alır.

Framing yöntemleri: Length-prefix (en yaygın), delimiter (HTTP'deki `\r\n\r\n`), fixed-size.

---

7. Graceful Shutdown

Doğru kapatma sırası:

1. `shutdown(SHUT_WR)` — FIN yolla, ama hâlâ **oku** (half-close)
2. Karşı taraf da FIN yollayana kadar bekle
3. `close()` — Tamamen kapat

Neden? Ani `close()`, tampondaki veriyi kaybettirebilir.

---

8. Timeout'lar

3 farklı timeout var:

- **Connect** — Bağlantı kurulurken
- **Read** — Veri beklerken (`SO_RCVTIMEO`)
- **Write** — Tampon doluysa (`SO_SNDTIMEO`)

Python'da kolay yol: `sock.settimeout(5.0)`

Uzun bağlantılarda: Uygulama seviyesi heartbeat (ping/pong) gerekir, TCP keepalive 2 saat bekler.

---

9. SIGPIPE — Sessiz Katil

Kapalı bir socket'e yazmak Linux'ta **SIGPIPE** yollar, default davranış **process'i öldürür**.

Çözüm:

```
signal.signal(signal.SIGPIPE, signal.SIG_IGN)   # veya
sock.send(data, socket.MSG_NOSIGNAL)
```

---

10. Hata Yönetimi

Hata	Anlamı	Ne yap?
`EINTR`	Signal kesti	Retry
`EAGAIN`/`EWOULDBLOCK`	Veri yok (non-blocking)	poll/select'e dön
`ECONNRESET`	RST aldın	Bağlantı ölü, temizle
`EPIPE`	Karşı taraf kapattı	SIGPIPE'i yoksay + kapat
`EMFILE`	fd limiti doldu	Backoff + log
`EADDRINUSE`	Port kullanımda	`SO_REUSEADDR`

---

11. Performans için Temel Kurallar

1. **Tamponu yeniden kullan** — Her mesajda allocate etme
2. **Batching yap** — Küçük yazımları birleştir
3. **`TCP_NODELAY`** — Etkileşimli uygulamalarda kapat Nagle'ı
4. **Event-driven** — >10K bağlantıda thread'den vazgeç
5. **fd limitlerini artır** — `ulimit -n 100000`
6. **Connection pooling** — Kısa bağlantı yerine yeniden kullan

---

12. Debug Araçları

Araç	Ne işe yarar
`ss -tanp`	Socket durumları, kuyruklar
`netstat -s`	Protokol istatistikleri (retrans, drop)
`tcpdump`	Paket seviyesi inceleme
`lsof -i`	Hangi process hangi socket'i kullanıyor

Faydalı tek satırlar:

```
ss -s                                   # özet
ss -tan state time-wait | wc -l         # TIME_WAIT sayısı
```

Özet

1. **Socket = fd + kernel nesnesi**, sihir değil
2. `accept()` **yeni fd döner** (dinleyen socket değil)
3. `recv()` **tüm mesajı getirmez** → kendi framing'ini yap (length-prefix)
4. `SO_REUSEADDR` olmadan yoğun sunucu yazılmaz
5. `TCP_NODELAY` etkileşimli uygulamada şart (Nagle tuzağı)
6. **CLOSE_WAIT yığılıyorsa** uygulama bug'ı var
7. **Graceful shutdown**: `shutdown(SHUT_WR)` → bekle → `close()`
8. `ss`/`tcpdump` olmadan debug yapılmaz

ARQ (Automatic Repeat reQuest) Protokolü Nedir?

ARQ (Otomatik Tekrar İsteği), güvenilmez bir kanal üzerinden veri iletirken **kayıp ve hatalı paketleri otomatik olarak tespit edip yeniden gönderen** bir hata kontrol mekanizmasıdır. TCP'nin de temelini oluşturur.

Neden Gerekli?

UDP gibi güvenilmez bir protokolde:

- Paketler **kaybolabilir** (network drop)
- Paketler **bozulabilir** (bit error)
- Paketler **sırasız** gelebilir
- Paketler **çoğaltılabilir** (duplicate)

ARQ bu sorunları çözmek için 3 temel araç kullanır:

1. **Sequence Number** – Her pakete sıra numarası
2. **ACK/NACK** – Alındı onayı / olumsuz onay
3. **Timeout & Retransmission** – Süre aşımında tekrar gönderim

---

ARQ Türleri

1. Stop-and-Wait ARQ

En basit ARQ. Gönderici **bir paket yollar, ACK bekler**, ACK gelince sıradakini yollar.

```
Gönderici          Alıcı
   |--- DATA(seq=0) -->|
   |<---- ACK(0) ------|
   |--- DATA(seq=1) -->|
   |<---- ACK(1) ------|
```

Avantajı: Basit, az bellek kullanır.
Dezavantajı: Kanal boş bekler → **düşük verim (throughput)**.
Örnek: RTT=100ms ise saniyede en fazla 10 paket.

Bizim kodda kullandığımız tam olarak bu! Her paket için tek ACK bekliyoruz.

---

2. Go-Back-N (GBN) ARQ

Gönderici **N paketlik pencere** kadar ACK beklemeden yollar. Bir paket kaybolursa, **o andan itibaren tüm sonraki paketler yeniden gönderilir**.

```
Gönderici          Alıcı
   |--- DATA(0) -->|
   |--- DATA(1) -->|   <- kayboldu
   |--- DATA(2) -->|
   |<---- ACK(0) --|
   |<---- NAK(1) --|   <- alıcı 1'i istiyor
   |--- DATA(1) -->|   <- hepsi tekrar
   |--- DATA(2) -->|
   |--- DATA(3) -->|
   |<---- ACK(3) --|   <- kümülatif ACK
```

- Alıcı **kümülatif ACK** yollar (`ACK(n)` = "n ve öncesini aldım").
- Kayıp olsa bile sonraki paketler gelir ama **atılır** (buffer'da sıra dışı kabul edilmez).
- **Avantajı:** Alıcı basit, bellek tutmaz.
- **Dezavantajı:** Kayıpta çok fazla gereksiz yeniden gönderim → bant genişliği israfı.

---

3. Selective Repeat (SR) ARQ

Sadece **kaybolan paket** yeniden gönderilir; diğerleri kabul edilir.

```
Gönderici          Alıcı
   |--- DATA(0) -->|
   |--- DATA(1) -->|   <- kayboldu
   |--- DATA(2) -->|   <- alıcı buffer'a alır
   |--- DATA(3) -->|   <- alıcı buffer'a alır
   |<---- ACK(0) --|
   |<---- NAK(1) --|
   |--- DATA(1) -->|   <- SADECE 1 tekrar
   |<---- ACK(1) --|
   |<---- ACK(2) --|
   |<---- ACK(3) --|
```

- Alıcı **sıra dışı gelen paketleri buffer'da** tutar.
- Her paket için **ayrı ACK** gerekir (kümülatif değil).
- **Avantajı:** En verimli yöntem, az yeniden gönderim.

- **Dezavantajı:** Alıcı tarafı karmaşık, her iki tarafta pencere yönetimi gerekir.

Karşılaştırma Tablosu

Özellik	Stop-and-Wait	Go-Back-N	Selective Repeat
Pencere boyutu (gönderici)	1	N	N
Pencere boyutu (alıcı)	1	1	N
Verim	Çok düşük	Orta	Yüksek
ACK tipi	Tek	Kümülatif	Tek tek
Alıcı belleği	Yok	Yok	N paket
Kayıpta retransmit	1 paket	Kayıptan sonrası hepsi	Sadece kayıp
Karmaşıklık	Kolay	Orta	Zor

ARQ'nun Temel Bileşenleri Detaylı

Sequence Number (Sıra Numarası)

Her DATA paketine artan bir numara verilir. Alıcı bunu kullanarak:

- **Kayıp** tespit eder (`seq=5` beklerken `seq=7` gelirse 6 kayıp)
- **Duplicate** tespit eder (aynı seq iki kez geldiyse atar)
- **Sıralamayı** düzeltir (buffer'da sıraya koyar)

Seq numarası **sonlu** bir aralıkta döner (örn. $0–2^{32}-1$). Bu yüzden pencere boyutu dikkatli seçilmelidir (SR'de N ≤ seq_space/2).

ACK vs NACK

	ACK	NACK
Anlamı	"Aldım"	"Alamadım, tekrar gönder"
Tetikleyici	Başarılı teslim	Kayıp tespiti
Trafik	Her paket için	Sadece kayıpta
TCP kullanır mı?	✅ Evet	Genelde hayır (SACK var)

Bizim kodda ikisi de var:

- DATA alınınca → **ACK**
- Sıra boşluğu görülünce → **NACK**

Timeout (RTO - Retransmission Timeout)

ACK gelmezse ne kadar bekleyip tekrar göndereceğiz? Bu süre kritik:

- **Çok kısa** → gereksiz yeniden gönderimler (spurious retransmission)
- **Çok uzun** → yavaş kurtarma, düşük throughput

TCP bunu **dinamik** hesaplar (Jacobson/Karels algoritması):

```
SRTT   = (1−α)·SRTT + α·RTT_sample    (α = 1/8)
RTTVAR = (1−β)·RTTVAR + β·|SRTT − RTT_sample|  (β = 1/4)
RTO    = SRTT + 4·RTTVAR
```

Bizim kodda sabit `TIMEOUT = 0.5 sn` kullandık; gerçek protokollerde dinamiktir.

Pencere (Window)

ACK beklemeden kaç paket gönderilebileceğini belirler:

```
Throughput ≈ Window_Size / RTT
```

Örnek: 100 Mbps, RTT=100ms → BDP (Bandwidth-Delay Product) = 1.25 MB. Bu kadar veriyi "uçuşta" tutmak için pencere ≥ 1.25 MB olmalı, yoksa kanal boş kalır.

Gerçek Dünyada ARQ

Protokol	ARQ Türü	Kullanım
TCP	Selective Repeat + SACK	İnternet
HDLC	Go-Back-N / SR	WAN linkleri
WiFi (802.11 MAC)	Stop-and-Wait	Kablosuz
Bluetooth L2CAP	Stop-and-Wait	Kısa menzil
QUIC	SR benzeri	HTTP/3
TFTP	Stop-and-Wait	Basit dosya transferi
Bizim kod	Stop-and-Wait + NACK	Öğretici

Bizim Kodun ARQ Analizi

Kodda uyguladığımız:

Sequence Number — Her DATA'ya artan seq
ACK — Alıcı her DATA için ACK yollar
Timeout — `TIMEOUT = 0.5 sn`
Retransmission — Timeout'ta tekrar gönder
NACK — Sıra boşluğu görülünce bildir
Max retry — 5 denemede pes et (dead-letter)

Eksik olanlar (gelişmiş ARQ için):

Sliding window — Stop-and-Wait olduğu için yavaş
Checksum/CRC — Bozuk paket tespiti yok
Dinamik RTO — Sabit timeout
Sıralı teslim — Buffer'da sıraya koyma yok
Duplicate filtresi — Aynı seq iki kez gelirse iki kez işlenir
Flow control — Alıcı yavaşsa gönderici ne yapmalı?
Congestion control — Ağ tıkanıksa ne yapmalı?

ARQ, güvenilmez bir kanalı güvenilir hale getirmek için kullanılan **"al–onayla–tekrar dene"** mekanizmasıdır. Üç ana varyantı vardır:

- **Stop-and-Wait** → basit ama yavaş
- **Go-Back-N** → orta verim, basit alıcı
- **Selective Repeat** → yüksek verim, karmaşık

> TCP, WiFi MAC, Bluetooth ve QUIC gibi birçok protokol ARQ'yu temel alır. Bizim UDP chat kodumuz **Stop-and-Wait ARQ + NACK**'in çalışan bir örneğidir.

---

4. UDP ile Basit Gönderim (Aynı Sistem Üzerinde)

Şimdi, aynı chat sistemini UDP protokolü ile yeniden tasarlayacağız. UDP'nin bağlantısız (connectionless) yapısını ve güvenilmezliğini gözlemleyeceğiz.

4.1. UDP Chat Uygulaması - Mimarisi

UDP'de "bağlantı" kavramı yoktur. Herkes birbirine doğrudan mesaj gönderebilir. Bu nedenle, basit bir UDP sohbetinde genellikle bir "sunucu" kavramı yoktur; herkes hem gönderici hem alıcıdır (peer-to-peer). Ancak karşılaştırma için, merkezi bir sunucu üzerinden yapalım:

- **UDP Sunucu:** Gelen verileri dinler ve gelen her paketi, paketi gönderen hariç tüm kayıtlı istemcilere iletir.
- **UDP İstemci:** Kullanıcı girdisini alır, paket haline getirip sunucuya gönderir ve aynı anda sunucudan gelen paketleri dinler.

4.2. UDP Sunucu Kodu (udp_server.py)

```
import socket

HOST = '127.0.0.1'
PORT = 12345
BUFFER_SIZE = 1024

# UDP soketi oluştur
server = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
server.bind((HOST, PORT))
print(f"UDP Sunucu {HOST}:{PORT} adresinde çalışıyor...")
```

```
# Bağlı istemcilerin adreslerini tut
clients = set()

while True:
    try:
        # Veriyi ve gönderen adresini al (recvfrom)
        data, client_addr = server.recvfrom(BUFFER_SIZE)

        # Yeni istemci ise listeye ekle
        if client_addr not in clients:
            clients.add(client_addr)
            print(f"Yeni istemci bağlandı: {client_addr}")
            # Katılım mesajını diğer herkese gönder
            join_msg = f"Yeni kullanıcı katıldı!
({client_addr})".encode('utf-8')
            for c in clients:
                if c != client_addr:
                    server.sendto(join_msg, c)

        # Mesajı gönderen hariç tüm istemcilere ilet
        print(f"{client_addr} mesajı: {data.decode('utf-8')}")
        for c in clients:
            if c != client_addr:
                server.sendto(data, c)

    except KeyboardInterrupt:
        print("\nSunucu kapatılıyor...")
        break
    except Exception as e:
        print(f"Hata: {e}")

server.close()
```

4.3. UDP İstemci Kodu (udp_client.py)

```
import socket
import threading
import time

HOST = '127.0.0.1'
PORT = 12345
BUFFER_SIZE = 1024

def receive_messages(client_socket):
    """Sunucudan gelen UDP paketlerini dinler."""
    while True:
        try:
            data, server_addr = client_socket.recvfrom(BUFFER_SIZE)
            print(f"\n{data.decode('utf-8')}")
            print("> ", end="", flush=True)  # Girdi istemini yeniden
göster
```

```
            except:
                break

    def start_udp_client():
        # UDP soketi oluştur
        client = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)

        # Alıcı thread'i başlat
        receive_thread = threading.Thread(target=receive_messages, args=
    (client,))
        receive_thread.daemon = True
        receive_thread.start()

        # Kullanıcı adı al
        nickname = input("Kullanıcı adınız: ")

        # Hoş geldin mesajını sunucuya gönder (bağlantı kurma amaçlı)
        client.sendto(f"{nickname} sohbete katıldı!".encode('utf-8'), (HOST,
    PORT))
        time.sleep(0.1)  # Mesajın gitmesi için kısa bekleme

        print(f"UDP Sohbetine hoş geldiniz! (Sunucu: {HOST}:{PORT})")
        print("Mesajlarınızı yazın (Çıkmak için 'quit' yazın):")

        while True:
            message = input("> ")
            if message.lower() == 'quit':
                client.sendto(f"{nickname} sohbetten ayrıldı.".encode('utf-8'),
    (HOST, PORT))
                break

            # Mesajı kullanıcı adı ile birlikte gönder
            full_message = f"{nickname}: {message}"
            client.sendto(full_message.encode('utf-8'), (HOST, PORT))

        client.close()

    if __name__ == "__main__":
        start_udp_client()
```

---

5. TCP Chat vs UDP Chat Karşılaştırmalı Analiz

Özellik	TCP Chat	UDP Chat
Bağlantı Kurulumu	3-way handshake gerekli. Her istemci ayrı bir soket ile sunucuya bağlanır.	Bağlantı kurulumu yok. `sendto()` ile doğrudan gönderim.
İstemci Takibi	Sunucu, `clients` listesinde her istemci için ayrı bir socket tutar.	Sunucu, `clients` set'inde IP+Port ikililerini (adres) tutar.

Özellik	TCP Chat	UDP Chat
Veri Alımı	`recv()` (blocking). Bağlantı koparsa `recv()` hata fırlatır.	`recvfrom()` (blocking). Bağlantı kopması diye bir durum yok.
Paket Kaybı	Hiçbir mesaj kaybolmaz. TCP garantiler.	Mesajlar kaybolabilir. Örneğin ağ tıkanıklığında paket düşebilir.
Mesaj Sırası	Mesajlar gönderim sırasıyla alınır.	Mesajlar sırasız gelebilir.
Hata Yönetimi	Bağlantı koptuğunda `remove_client()` ile temizlik yapılır.	Bağlantı kopması tespiti yok. Kayıp paketler fark edilmez.
Performans	Daha yavaş, daha fazla overhead (kontrol mekanizmaları).	Daha hızlı, minimal overhead.
Kod Karmaşıklığı	Orta seviye (thread yönetimi, bağlantı kontrolü).	Daha basit (doğrudan gönderim/alım).

---

6. UDP Chat Uygulaması ile Yapılabilecek Deneyler

1. **Paket Kaybını Gözlemleme:**
   - Ağ trafiğini yoğunlaştırarak (örneğin büyük dosya indirirken) UDP chat kullanın.
   - Bazı mesajların karşı tarafa ulaşmadığını gözlemleyin.
2. **Sıra Bozukluğunu Test Etme:**
   - Hızlıca ardışık mesajlar gönderin.
   - UDP'de bazen mesajların farklı sırada alındığını fark edin.
3. **Bağlantı Kopma Tespiti Eksikliği:**
   - İstemciyi kapatın ama sunucu bunu fark etmez.
   - Sunucu, kapalı istemciye mesaj göndermeye çalışır (bu hata fırlatmaz, sadece paket kaybolur).

---

7. Oyun Geliştirme Perspektifinden Çıkarımlar

Kavram	TCP Chat	UDP Chat	Oyundaki Yansıması
Blocking I/O	`recv()` programı bekletir.	`recvfrom()` programı bekletir.	Oyun döngüsünde (game loop) blocking I/O kullanılmaz. Non-blocking veya ayrı thread ile çözülür.
Connection Tracking	Socket listesi ile takip.	IP+Port listesi ile takip.	Oyun sunucuları, her oyuncuyu benzersiz `playerID` ile takip eder.
Reliability	Tüm mesajlar ulaşır.	Mesajlar kaybolabilir.	Hareket verileri UDP (kayıp tolere edilir), envanter

Kavram	TCP Chat	UDP Chat	Oyundaki Yansıması
			işlemleri TCP (kayıp kabul edilmez).
State Management	Bağlantı durumu (connected/disconnected) net.	"Bağlantı" kavramı yok.	UDP'de oyuncuların hala aktif olup olmadığını "heartbeat" (kalp atışı) ile kontrol ederiz.

İleri Çalışma

1. **TCP Chat'e Non-blocking Ekleme:**
   - TCP sunucusunu, her istemci için thread açmak yerine `select()` kullanarak tek thread'de birden fazla istemciyi yönetecek şekilde dönüştürün.
2. **UDP Chat'e "Güvenilirlik" Katmanı Ekleyin:**
   - Her mesaja bir sıra numarası (sequence number) ekleyin.
   - Kayıp mesajları tespit edip yeniden gönderme isteği (ACK/NACK) mekanizması tasarlayın. (Bu, UDP üzerinde TCP benzeri bir protokol oluşturmak demektir.)
3. **Her İki Protokolü Bir Arada Kullanma:**
   - TCP bağlantısı üzerinden "hoş geldin" ve "kimlik doğrulama" yapın.
   - Başarılı girişten sonra, UDP üzerinden oyun verilerini (konum, atış) gönderin.

Blocking I/O, basit uygulamalarda (chat gibi) kullanıcı sayısı az olduğunda tercih edilir ve thread ile çözülebilir. Non-blocking I/O, yüksek performanslı sunucular için (oyun sunucuları) gereklidir.

##UDP Güvevinilirlik katmanı ekleyelim

UDP Chat'e Güvenilirlik Katmanı Ekleme

UDP üzerine TCP benzeri bir güvenilirlik katmanı ekleyeceğiz. Temel bileşenler:

1. **Sequence Number (Sıra Numarası)**: Her pakete artan bir numara
2. **ACK (Acknowledgement)**: Alındı onayı
3. **NACK/Timeout ile Yeniden Gönderim**: Kayıp tespiti ve retransmission
4. **Mesaj Bütünlüğü**: Header + Payload yapısı

Paket Formatı

```
[MSG_TYPE:1][SEQ:4][ACK:4][PAYLOAD_LEN:2][PAYLOAD:N]
```

- `MSG_TYPE`: 0=DATA, 1=ACK, 2=NACK, 3=JOIN, 4=LEAVE

- `SEQ`: Bu paketin sıra numarası
- `ACK`: Onaylanan sequence numarası
- `PAYLOAD_LEN`: Payload uzunluğu (big-endian)

---

1. Ortak Protokol Modülü (`protocol.py`)

```
import struct

# Mesaj tipleri
MSG_DATA = 0
MSG_ACK = 1
MSG_NACK = 2

# Header: type(1) + seq(4) + ack(4) + payload_len(2) = 11 byte
HEADER_FORMAT = '!BIIH'
HEADER_SIZE = struct.calcsize(HEADER_FORMAT)  # 11


def pack_message(msg_type, seq, ack, payload: bytes) -> bytes:
    """Mesajı header + payload olarak paketle."""
    header = struct.pack(HEADER_FORMAT, msg_type, seq, ack, len(payload))
    return header + payload


def unpack_message(data: bytes):
    """Gelen paketi çöz. (type, seq, ack, payload) döner."""
    if len(data) < HEADER_SIZE:
        return None
    msg_type, seq, ack, payload_len = struct.unpack(
        HEADER_FORMAT, data[:HEADER_SIZE]
    )
    payload = data[HEADER_SIZE:HEADER_SIZE + payload_len]
    return msg_type, seq, ack, payload
```

---

2. Güvenilir UDP Sunucu (`server.py`)

Sunucu tarafında her istemci için ayrı bir "yeniden gönderim kuyruğu" tutuyoruz. İstemciden ACK gelmezse timeout sonrası tekrar gönderiyoruz.

```
import socket
import threading
import time
from protocol import (
    pack_message, unpack_message,
    MSG_DATA, MSG_ACK, MSG_NACK, HEADER_SIZE
)

HOST = '127.0.0.1'
```

```
PORT = 12345
BUFFER_SIZE = 2048

# Zaman aşımı ve yeniden gönderim ayarları
TIMEOUT = 0.5       # saniye
MAX_RETRIES = 5

server = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
server.bind((HOST, PORT))
print(f"[SERVER] UDP Sunucu {HOST}:{PORT} adresinde çalışıyor...")

clients = {}          # addr -> next_seq (gönderilecek sıradaki seq)
lock = threading.Lock()

# Yeniden gönderim kuyruğu: (client_addr) -> {seq: (packet_bytes, retries,
last_sent)}
pending = {}


def get_next_seq(addr):
    with lock:
        seq = clients.get(addr, 1)
        clients[addr] = seq + 1
        return seq


def reliable_send(addr, msg_type, payload: bytes):
    """Mesajı seq numarası ile gönder ve ACK için kuyruğa al."""
    seq = get_next_seq(addr)
    packet = pack_message(msg_type, seq, 0, payload)

    if addr not in pending:
        pending[addr] = {}
    pending[addr][seq] = [packet, 0, time.time()]

    server.sendto(packet, addr)


def retransmit_loop():
    """Periyodik olarak ACK'i gelmeyen paketleri yeniden gönder."""
    while True:
        time.sleep(0.1)
        now = time.time()
        for addr in list(pending.keys()):
            for seq in list(pending[addr].keys()):
                packet, retries, last = pending[addr][seq]
                if now - last >= TIMEOUT:
                    if retries >= MAX_RETRIES:
                        print(f"[SERVER] {addr} seq={seq} için max deneme
aşıldı, bırakılıyor.")
                        del pending[addr][seq]
                        continue
                    print(f"[SERVER] Yeniden gönderim -> {addr} seq={seq}
(deneme {retries+1})")
```

```
                    server.sendto(packet, addr)
                    pending[addr][seq][1] = retries + 1
                    pending[addr][seq][2] = now


def handle_ack(addr, ack_seq):
    """Gelen ACK'i işle, kuyruktan çıkar."""
    if addr in pending and ack_seq in pending[addr]:
        del pending[addr][ack_seq]
        print(f"[SERVER] ACK alındı: {addr} seq={ack_seq}")


def broadcast(sender_addr, msg_type, payload):
    """Gönderen hariç tüm istemcilere güvenilir şekilde ilet."""
    for c in list(clients.keys()):
        if c != sender_addr:
            reliable_send(c, msg_type, payload)


def main():
    threading.Thread(target=retransmit_loop, daemon=True).start()

    while True:
        try:
            data, addr = server.recvfrom(BUFFER_SIZE)
            parsed = unpack_message(data)
            if not parsed:
                continue
            msg_type, seq, ack, payload = parsed

            # ACK paketi -> gönderim kuyruğundan çıkar
            if msg_type == MSG_ACK:
                handle_ack(addr, seq)
                continue

            # DATA paketi geldi -> kendi ACK'imizi gönder
            if msg_type == MSG_DATA:
                ack_pkt = pack_message(MSG_ACK, seq, seq, b"")
                server.sendto(ack_pkt, addr)

                text = payload.decode('utf-8', errors='ignore')
                print(f"[SERVER] {addr} (seq={seq}): {text}")

                # Yeni istemci mi?
                if addr not in clients:
                    clients[addr] = 1
                    print(f"[SERVER] Yeni istemci: {addr}")
                    join_msg = f"*** Yeni kullanıcı katıldı! ({addr})
***".encode()
                    broadcast(addr, MSG_DATA, join_msg)

                # Diğerlerine ilet
                broadcast(addr, MSG_DATA, payload)
```

```
            except KeyboardInterrupt:
                print("\n[SERVER] Kapatılıyor...")
                break
            except Exception as e:
                print(f"[SERVER] Hata: {e}")

        server.close()


if __name__ == "__main__":
    main()
```

---

3. Güvenilir UDP İstemci (`client.py`)

İstemci iki tarafı da yönetir:

- **Gönderirken**: ACK bekle, gelmezse timeout'ta tekrar gönder
- **Alırken**: Her DATA için ACK yolla, sıra numarası boşluğu görürse NACK (opsiyonel)

```
import socket
import threading
import time
from protocol import (
    pack_message, unpack_message,
    MSG_DATA, MSG_ACK, MSG_NACK
)

HOST = '127.0.0.1'
PORT = 12345
BUFFER_SIZE = 2048

TIMEOUT = 0.5
MAX_RETRIES = 5


class ReliableUDPClient:
    def __init__(self, nickname):
        self.nickname = nickname
        self.sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
        self.server = (HOST, PORT)

        self.send_seq = 1
        self.pending = {}       # seq -> [packet, retries, last_sent]
        self.pending_lock = threading.Lock()

        self.expected_seq = {}  # addr -> beklenen sonraki seq (NACK için)
        self.running = True

    def send_reliable(self, payload: bytes):
        """Payload'ı seq ile gönder, ACK gelene kadar takip et."""
```

```
        with self.pending_lock:
            seq = self.send_seq
            self.send_seq += 1
            packet = pack_message(MSG_DATA, seq, 0, payload)
            self.pending[seq] = [packet, 0, time.time()]
        self.sock.sendto(packet, self.server)
        return seq

    def retransmit_loop(self):
        while self.running:
            time.sleep(0.1)
            now = time.time()
            with self.pending_lock:
                for seq in list(self.pending.keys()):
                    packet, retries, last = self.pending[seq]
                    if now - last >= TIMEOUT:
                        if retries >= MAX_RETRIES:
                            print(f"\n[!] seq={seq} için max deneme
aşıldı.")
                            del self.pending[seq]
                            continue
                        self.sock.sendto(packet, self.server)
                        self.pending[seq][1] = retries + 1
                        self.pending[seq][2] = now
                        print(f"\n[RETRANSMIT] seq={seq} tekrar
gönderildi.")

    def receive_loop(self):
        while self.running:
            try:
                data, addr = self.sock.recvfrom(BUFFER_SIZE)
                parsed = unpack_message(data)
                if not parsed:
                    continue
                msg_type, seq, ack, payload = parsed

                if msg_type == MSG_ACK:
                    with self.pending_lock:
                        if seq in self.pending:
                            del self.pending[seq]
                            print(f"\n[ACK] seq={seq} onaylandı.")
                    print("> ", end="", flush=True)
                    continue

                if msg_type == MSG_DATA:
                    # Her DATA için ACK yolla
                    ack_pkt = pack_message(MSG_ACK, seq, seq, b"")
                    self.sock.sendto(ack_pkt, addr)

                    # NACK kontrolü: beklenen seq değilse kayıp var
                    expected = self.expected_seq.get(addr, 1)
                    if seq > expected:
                        # Boşluk: eksik seq'leri NACK'le bildir
                        for missing in range(expected, seq):
```

```
                            nack = pack_message(MSG_NACK, missing, missing,
b"")
                            self.sock.sendto(nack, addr)
                            print(f"\n[NACK] eksik seq={missing}
bildirildi.")
                    self.expected_seq[addr] = seq + 1

                    print(f"\n{payload.decode('utf-8', errors='ignore')}")
                    print("> ", end="", flush=True)

                if msg_type == MSG_NACK:
                    # Karşı taraf eksik paketi istiyor -> yeniden gönder
                    with self.pending_lock:
                        if seq in self.pending:
                            self.sock.sendto(self.pending[seq][0], addr)
                            print(f"\n[NACK alındı] seq={seq} yeniden
gönderiliyor.")
                    print("> ", end="", flush=True)

            except OSError:
                break


def main():
    nickname = input("Kullanıcı adınız: ")
    client = ReliableUDPClient(nickname)

    threading.Thread(target=client.receive_loop, daemon=True).start()
    threading.Thread(target=client.retransmit_loop, daemon=True).start()

    join = f"*** {nickname} sohbete katıldı ***".encode()
    client.send_reliable(join)
    time.sleep(0.2)

    print(f"Güvenilir UDP Sohbeti (Sunucu: {HOST}:{PORT})")
    print("Çıkmak için 'quit' yazın.\n")

    while True:
        msg = input("> ")
        if msg.lower() == 'quit':
            leave = f"*** {nickname} ayrıldı ***".encode()
            client.send_reliable(leave)
            time.sleep(0.5)
            break

        full = f"{nickname}: {msg}".encode('utf-8')
        client.send_reliable(full)

    client.running = False
    client.sock.close()
```

```
if __name__ == "__main__":
    main()
```

Nasıl Çalışıyor?

Sıra Numarası (Sequence Number)

- Her istemcinin **kendi gönderim seq'i** var (1'den başlar, artar).
- Sunucu da her istemciye ayrı seq atar.
- Bu sayede gelen paket sıraya dizilebilir ve kayıp tespit edilebilir.

ACK Mekanizması

- DATA alındığında alıcı aynı seq numarasını **ACK** olarak geri yollar.
- Gönderici ACK gelene kadar paketi `pending` sözlüğünde tutar.
- ACK gelirse `pending`'den silinir.

Timeout + Retransmission

- `retransmit_loop` her 100 ms'de bir `pending`'i kontrol eder.
- `TIMEOUT = 0.5 sn` geçmiş ve ACK gelmemiş paketleri tekrar gönderir.
- `MAX_RETRIES = 5` sonrası pes eder (dead-letter).

NACK (İsteğe Bağlı İyileştirme)

- Alıcı beklediği `seq`'den büyük bir numara görürse (`seq > expected`), arada kayıp olanları **NACK** ile bildirir.
- Gönderici NACK alınca ilgili paketi anında yeniden gönderir (timeout beklemeden → daha hızlı kurtarma).

Test Senaryosu

1. **Sunucuyu başlat**: `python server.py`
2. **İki istemci başlat**: `python client.py` (iki terminal)
3. **Normal sohbet**: Mesajlar karşılıklı gider, ACK log'larını görürsünüz.
4. **Kayıp simülasyonu**: `client.py`'de `sendto` öncesi rastgele `drop` ekleyin:

```
import random
if random.random() < 0.3:  # %30 paket kaybı
    return seq  # gönderme
```

→ Timeout sonrası yeniden gönderim devreye girer.

Sınırlamalar ve Geliştirmeler

Konu	Mevcut Durum	Geliştirme
Sıralama	`expected_seq` sadece NACK için	Gelen paketleri tamponla, sıralı teslim et
Pencereleme	Tek paket ACK bekle (stop-and-wait)	Sliding window ile throughput artır
Checksum	Yok	Payload'a CRC32 ekle
Congestion control	Yok	AIMD benzeri bir mekanizma ekle
Duplicate	Sadece ACK'lenir	Duplicate'leri tespit edip filtrele

Bu yapı, UDP üzerinde **stop-and-wait ARQ** protokolünün çalışan bir örneğidir; sliding window ve checksum eklenirse gerçek TCP'ye çok yaklaşır.